

ROS2

DAY 07 – Launch Files

Objective

By the end of this manual, you will be able to:

- Understand what a launch file is and why it is needed
 - Run multiple ROS 2 nodes using a single command
 - Pass parameters using launch files
 - Understand the basic structure of a Python launch file
 - Create and run your first ROS 2 launch file
-

Step 0 – Pre-Requisites

Before starting, make sure:

- ROS 2 is installed (Humble / Foxy / Rolling)
 - You know how to:
 - Create a ROS 2 workspace
 - Create a Python package
 - Run a node using `ros2 run`
 - You have already worked with nodes, topics, services, and parameters (Days 1–6)
-

Step 1 – What Is a Launch File?

A launch file in ROS 2 is a configuration file used to start and manage multiple ROS 2 nodes together using a single command.

It allows you to define which nodes run, how they are configured (parameters), how topics are remapped, and how the system starts, instead of launching each node manually in separate terminals.

A launch file is used to:

- Start one or more ROS 2 nodes
- Configure parameters
- Remap topics
- Set node names
- Control execution from one command

Without a launch file

You open multiple terminals:

```
ros2 run pkg node1  
ros2 run pkg node2  
ros2 run pkg node3
```

With a launch file

You run a single command:

```
ros2 launch pkg my_launch.py
```

Step 2 – Why Launch Files Are Important

Launch files are essential in real robots because:

- Robots run many nodes together
- Parameters must be set before startup
- Nodes must start in a controlled way
- Experiments and demos must be repeatable

In industry, robots are never started node-by-node manually.

Step 3 – Types of Launch Files in ROS 2

ROS 2 supports:

- Python launch files (.launch.py) ✓ Most common

- XML launch files
- YAML launch files

You will use Python launch files because they are:

- Flexible
 - Programmable
 - Industry standard
-

Implementation – Step-by-Step

What We Will Build

We will:

- Create two simple Python nodes (Publisher + Subscriber)
 - Launch both nodes using a single launch file
 - Pass a parameter from the launch file
 - Verify execution using ROS 2 CLI tools
-

Step 1 – Create Workspace

Open a new terminal:

```
mkdir -p DAY-07/ros2_ws/src
cd DAY-07/ros2_ws
colcon build
source install/setup.bash
```

This initializes a clean ROS 2 workspace.

Step 2 – Create a Python Package

```
cd src
ros2 pkg create launch_demo_pkg --build-type ament_python
```

Package structure:

```
launch_demo_pkg/  
├── launch_demo_pkg/  
├── package.xml  
└── setup.py
```

Step 3 – Create Two Simple Nodes

Step 3.1 – Navigate to Source Folder

```
cd launch_demo_pkg/launch_demo_pkg
```

Step 3.2 – Create Publisher Node

```
nano talker_node.py
```

Publisher code:

```
import rclpy  
from rclpy.node import Node  
from std_msgs.msg import String  
  
class TalkerNode(Node):  
    def __init__(self):  
        super().__init__('talker_node')  
        self.publisher = self.create_publisher(String, 'chatter', 10)  
        self.timer = self.create_timer(1.0, self.timer_callback)  
  
    def timer_callback(self):  
        msg = String()  
        msg.data = "Hello from Talker"  
        self.publisher.publish(msg)  
        self.get_logger().info(msg.data)  
  
def main(args=None):  
    rclpy.init(args=args)  
    node = TalkerNode()  
    rclpy.spin(node)  
    rclpy.shutdown()  
  
if __name__ == '__main__':  
    main()
```

Step 3.3 – Create Subscriber Node

```
nano listener_node.py
```

Subscriber code:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class ListenerNode(Node):
    def __init__(self):
        super().__init__('listener_node')
        self.subscription = self.create_subscription(
            String,
            'chatter',
            self.callback,
            10
        )

    def callback(self, msg):
        self.get_logger().info(f"Received: {msg.data}")

def main(args=None):
    rclpy.init(args=args)
    node = ListenerNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Step 4 – Register Nodes in setup.py

```
cd ..
nano setup.py
```

Update data_files: Add the path of launch file

```
data_files=[
    ('share/' + package_name + '/launch', ['launch/launch.py']),
]
```

Update entry_points:

```
entry_points={
    'console_scripts': [
        'talker_node = launch_demo_pkg.talker_node:main',
        'listener_node = launch_demo_pkg.listener_node:main',
    ],
},
```

Step 5 – Create Launch File

```
mkdir launch
nano launch/launch.py
```

Launch File

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='launch_demo_pkg',
            executable='talker_node',
            name='talker_from_launch'
        ),
        Node(
            package='launch_demo_pkg',
            executable='listener_node',
            name='listener_from_launch'
        )
    ])
```

Understanding This File (Conceptual)

LaunchDescription

- Container that holds everything to be launched

Node

- Describes one ROS 2 node
- Defines:
 - package
 - executable
 - node name
 - parameters
 - remappings

generate_launch_description()

- Mandatory function
 - ROS 2 calls this automatically
-

Step 6 – Build and Run

```
cd ~/DAY-07/ros2_ws
colcon build
source install/setup.bash
ros2 launch launch_demo_pkg launch.py
```

Expected Output

```
[talker_from_launch] Hello from Talker
[listener_from_launch] Received: Hello from Talker
```

Step 7 – Verify Using ROS 2 CLI

```
ros2 node list
/talker_from_launch
/listener_from_launch
ros2 topic list
/chatter
```

Step 8 – Passing Parameters via Launch File

Modify `talker_node.py`

```
self.declare_parameter('message', 'Hello from Launch')  
msg.data = self.get_parameter('message').value
```

Modify Launch File

```
parameters=[{'message': 'Hello passed from launch file'}]
```

Rebuild and run again.

Final Summary

We have learned:

- Launching multiple nodes
 - Using Python launch files
 - Running systems with one command
 - Passing parameters from launch files
 - Verifying execution using ROS 2 CLI
-